UFRPE	**Universidade Federal Rural de Pernambuco** **Unidade Acadêmica de Belo Jardim** **Engenharia de Controle e Automação**	
	Roteiro da Prática 7 – Protocolo de Comunicação Serial I2C	
Disciplina:	Microcontroladores e Microprocessadores e Engenharia de Sistemas Embarcados	
Professor:	Henrique Patriota	
Alunos:		
Bancada:		Data:

1. Introdução

O protocolo I2C (*Inter-Integrated Circuit*) é um barramento de comunicação serial desenvolvido pela Philips para facilitar a comunicação entre múltiplos dispositivos utilizando apenas duas linhas: SDA (*Serial Data*), responsável pela transmissão e recepção de dados, e SCL (*Serial Clock*), responsável pela sincronização da comunicação. Ele opera no modelo *master-slave*, onde um ou mais dispositivos mestres podem controlar vários dispositivos escravos. Cada escravo possui um endereço único, garantindo que ele só responda aos comandos direcionados a ele. Isso permite que vários dispositivos compartilhem o mesmo barramento sem conflitos.

O protocolo I2C suporta diferentes velocidades de operação, dependendo das necessidades do sistema. As velocidades mais comuns incluem *Standard Mode* (100 kHz), *Fast Mode* (400 kHz), Fast *Mode Plus* (1 MHz) e *High-Speed Mode* (3.4 MHz). Essa flexibilidade permite o uso do I2C em uma ampla variedade de aplicações, desde dispositivos de baixa potência até sistemas que exigem maior taxa de transferência de dados. A comunicação ocorre em pacotes de 8 bits e é iniciada pelo mestre, que envia um *start condition*, seguido pelo endereço do escravo e um bit indicando se a operação será de leitura ou escrita. O escravo responde com um a*cknowledge* (ACK), confirmando o recebimento dos dados. A transmissão prossegue até que um *stop condition* seja enviado para encerrar a comunicação. Devido à sua simplicidade e eficiência, o protocolo I2C é amplamente utilizado em sistemas embarcados para conectar sensores, memórias EEPROM, relógios de tempo real (RTC), displays e diversos outros periféricos.

Esta prática aborda o protocolo serial I2C para a comunicação entre diversos microcontroladores ATmega328P embarcados em plataforma Arduino, com um desses microcontroladores configurado como *Master* e os demais como *Slaves*. Adicionalmente, teremos a demonstração de uma decodificação de um barramento I2C usando um osciloscópio.

2. Hardware e Descrição da Prática

Na Figura 1, é mostrado o esquema das conexões que devem ser realizadas entre os microcontroladores com o barramento I2C. O terminal analógico A5 também recebe a conexão do barramento *Serial Clock* (SCL), enquanto o terminal analógico A4 recebe a conexão do barramento *Serial Data* (SDA). Como deve ser observado, cada dispositivo I2C conectado ao barramento deve compartilhar das mesmas conexões, ou seja, SDA conecta com SDA e SCL conecta com SCL. Por fim, devemos notar também que todos os microcontroladores devem estar na mesma referência, ou seja, com os GNDs compartilhados entre si.

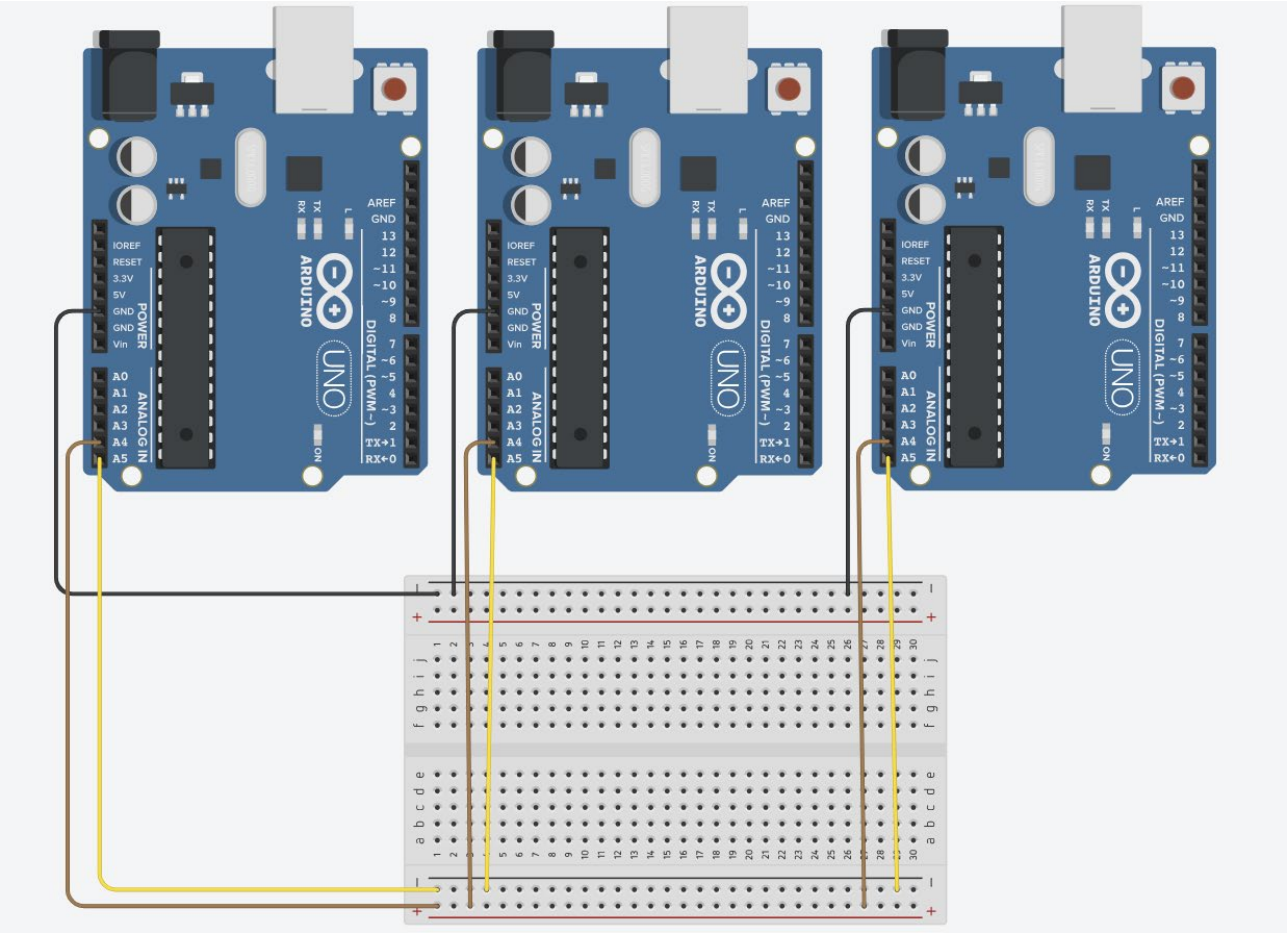


Figura 2 – Esquema de conexões entre três microcontroladores no mesmo barramento I2C.

Como um protocolo I2C padrão tem resolução de 7 bits, os dispositivos configurados como *Slaves* devem receber endereços entre 1 e 127, onde a base decimal também pode ser utilizada. Nesta prática, após as conexões entre os microcontroladores no barramento I2C, uma rotina de *software* deve ser implementada para o envio periódico de informações do *Master* para os *Slaves*.

3. Rotinas do software embarcado

Nesta prática, o *software* embarcado será implementado na IDE Arduino, com a distinção dos softwares usado pelo *Master* e pelos *Slaves*. A seguir, temos um exemplo de um software de um Master, que endereça uma variável de 8 bits, que está sendo incrementada em cada ciclo do loop infinito, para três escravos, com endereços 3, 17 e 25. Devemos notar que para cada escravo adicionado ao barramento, via software, deve-se definir a rotina para endereçamento e troca de informações entre o mestre e o escravo.

```
///Software do mestre do protocolo I2C
#include<Wire.h>// biblioteca do I2C

void setup(){
  Wire.begin();//Inicializa a comunicação serial I2C. O vazio () configura o dispositivo como mestre (sem endereço)
}

byte x=0; //cria variável x de 8 bits
byte x1=0;
byte x2=0;



void loop(){
  Wire.beginTransmission(25);//Escravo com endereço 25
  Wire.write("x = ");//Envia (escreve no barramento de dados) a string x =
  Wire.write(x);//Envia (escreve no barramento de dados)o valor de x
  Wire.endTransmission();// Finaliza a transmissão

  Wire.beginTransmission(17);//Escravo com endereço 17
  Wire.write("x1 = ");//Envia (escreve no barramento de dados) a string x =
  Wire.write(x1);//Envia (escreve no barramento de dados)o valor de x
  Wire.endTransmission();// Finaliza a transmissão

  Wire.beginTransmission(3);//Escravo com endereço 3
  Wire.write("x2 = ");//Envia (escreve no barramento de dados) a string x =
  Wire.write(x2);//Envia (escreve no barramento de dados)o valor de x
  Wire.endTransmission();// Finaliza a transmissão

  x++;
  x1++;
  x2++;

  delay(500);

```

Na sequência, temos um exemplo de *software* usado para o microcontrolador configurado como *Slave*, com endereço 17.

```ino
//Software do escravo, configurado como escravo

#include<Wire.h> //biblioteca do I2C

int x;//Variável que será recebida
char c; //Varial para leitura dos caracters enviados (a string x=)

void setup(){

  Wire.begin(17);//Inicializa a comunicação I2C definindo o dispositivo como escravo com endereço 8
  Wire.onReceive(evento);//Inicializa a recepção por meio da função evento
  Serial.begin(9600);//Inicializa a comunicação serial com taxa de 9600 bits/s
 }

void loop(){
  delay(100);
}

//definicação da função evento
void evento(int leitura){
  while(1<Wire.available()){// verifica se tem dados disponível no barramento SDA (serial data), que coloca ele em nível 0.
    c=Wire.read();//leitura no SDA
    Serial.print(c);
  }
  x=Wire.read();//leitura no SDA
  Serial.println(x);

}
```